

TP 9 : Apache kafka

Université / École :

- Université Ibn Tofail, Kénitra

Encadrement pédagogique :

- Pr. Hajar Filali

Rédigé par :

- Mohammed Jebrou
- Hajar Farid

Année universitaire :

- 2025-2026

Introduction à Apache Kafka

Apache Kafka est une plateforme de streaming distribuée conçue pour la collecte, le transport et le traitement de flux de données en temps réel. Développée à l'origine par LinkedIn puis intégrée à la fondation Apache, elle permet l'échange rapide et fiable de données entre différentes applications grâce à un système basé sur les producteurs (producer), les consommateurs (consumer) et les topics.

Dans ce travail pratique, nous allons installer et configurer Apache Kafka, puis découvrir ses principaux mécanismes de fonctionnement à travers la création de topics et l'échange de messages entre producteurs et consommateurs.

Objectifs du TP

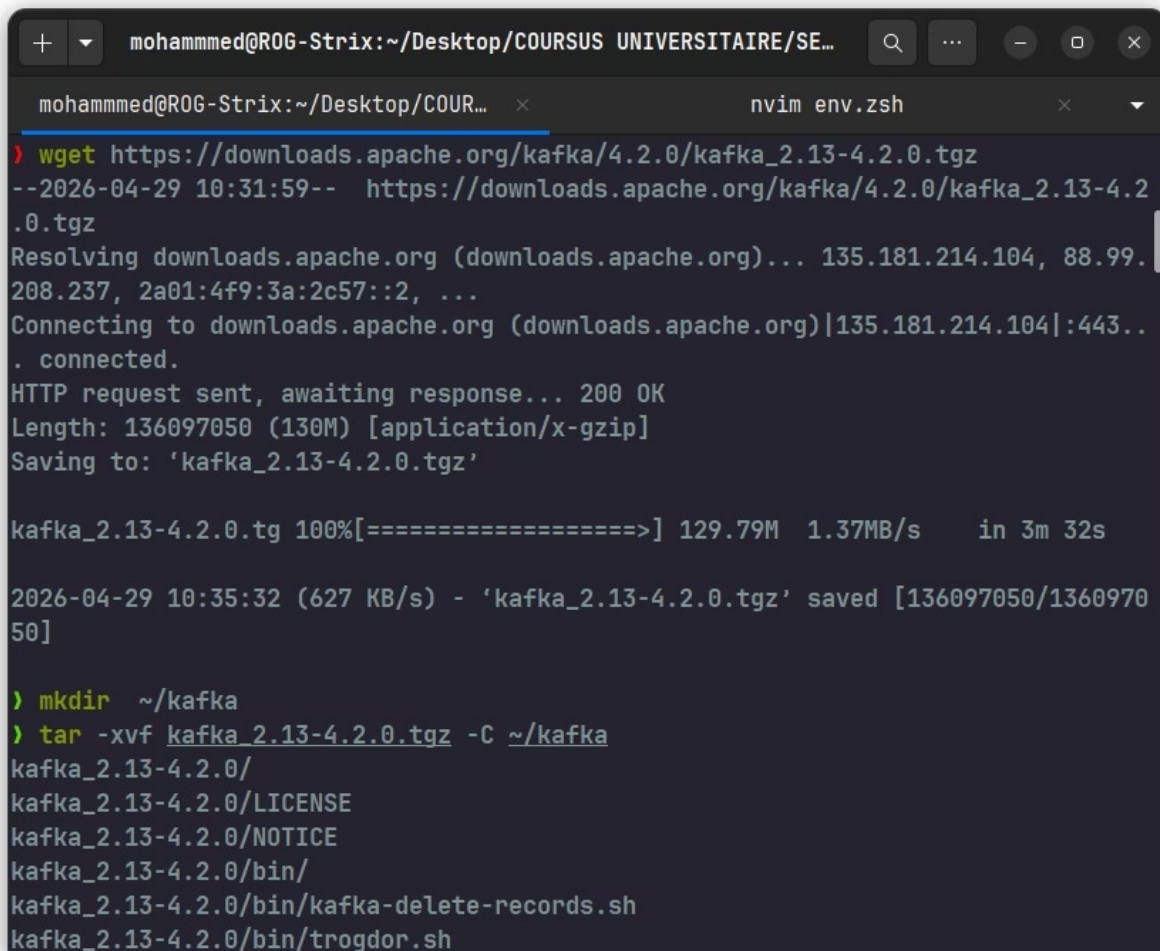
À l'issue de ce travail pratique, nous serons capables de :

- Installer et configurer Apache Kafka.
- Configurer les variables d'environnement nécessaires à son fonctionnement.
- Démarrer les services Kafka et leurs composants associés.
- Créer et administrer des topics Kafka.
- Produire et consommer des messages.
- Comprendre le fonctionnement des producteurs et des consommateurs.
- Manipuler les commandes essentielles d'administration de Kafka.
- Découvrir les principes fondamentaux du streaming de données en temps réel.

Installation du Kafka

1. Téléchargement et extraction des fichiers

Cette étape consiste à télécharger l'archive officielle d'Apache Kafka puis à l'extraire dans le répertoire de travail. Les fichiers obtenus contiennent les exécutables, les bibliothèques et les fichiers de configuration nécessaires au fonctionnement de Kafka.

A terminal window titled 'mohammed@ROG-Strix:~/Desktop/COURSUS UNIVERSITAIRE/SE...' with a tab for 'nvim env.zsh'. The terminal shows the execution of 'wget' to download 'kafka_2.13-4.2.0.tgz' from 'https://downloads.apache.org/kafka/4.2.0/'. The download progress is shown as 100% with a speed of 1.37MB/s. After the download, 'mkdir ~/kafka' is run, followed by 'tar -xvf kafka_2.13-4.2.0.tgz -C ~/kafka' to extract the files. The extracted files listed are 'kafka_2.13-4.2.0/', 'kafka_2.13-4.2.0/LICENSE', 'kafka_2.13-4.2.0/NOTICE', 'kafka_2.13-4.2.0/bin/', 'kafka_2.13-4.2.0/bin/kafka-delete-records.sh', and 'kafka_2.13-4.2.0/bin/trogdor.sh'.

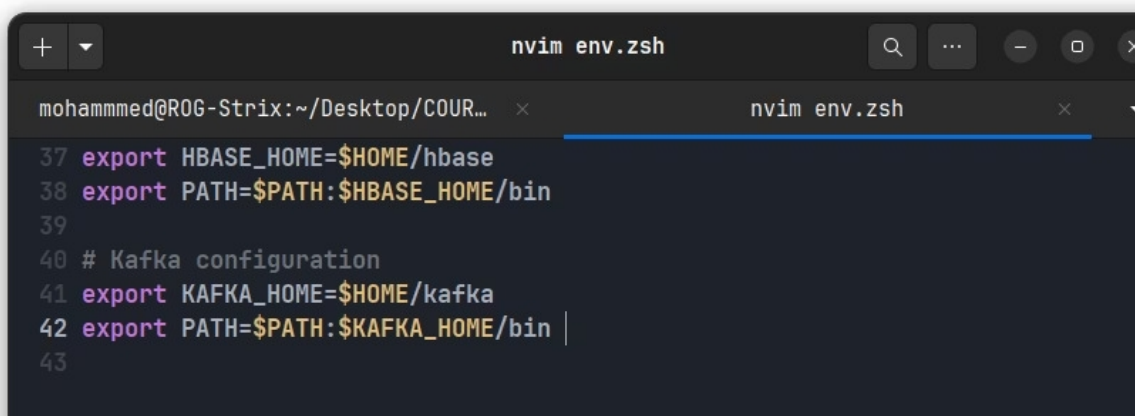
```
mohammed@ROG-Strix:~/Desktop/COURSUS UNIVERSITAIRE/SE...
nvim env.zsh
> wget https://downloads.apache.org/kafka/4.2.0/kafka_2.13-4.2.0.tgz
--2026-04-29 10:31:59-- https://downloads.apache.org/kafka/4.2.0/kafka_2.13-4.2.0.tgz
Resolving downloads.apache.org (downloads.apache.org)... 135.181.214.104, 88.99.208.237, 2a01:4f9:3a:2c57::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|135.181.214.104|:443..
. connected.
HTTP request sent, awaiting response... 200 OK
Length: 136097050 (130M) [application/x-gzip]
Saving to: 'kafka_2.13-4.2.0.tgz'

kafka_2.13-4.2.0.tg 100%[=====] 129.79M  1.37MB/s   in 3m 32s

2026-04-29 10:35:32 (627 KB/s) - 'kafka_2.13-4.2.0.tgz' saved [136097050/136097050]

> mkdir ~/kafka
> tar -xvf kafka_2.13-4.2.0.tgz -C ~/kafka
kafka_2.13-4.2.0/
kafka_2.13-4.2.0/LICENSE
kafka_2.13-4.2.0/NOTICE
kafka_2.13-4.2.0/bin/
kafka_2.13-4.2.0/bin/kafka-delete-records.sh
kafka_2.13-4.2.0/bin/trogdor.sh
```

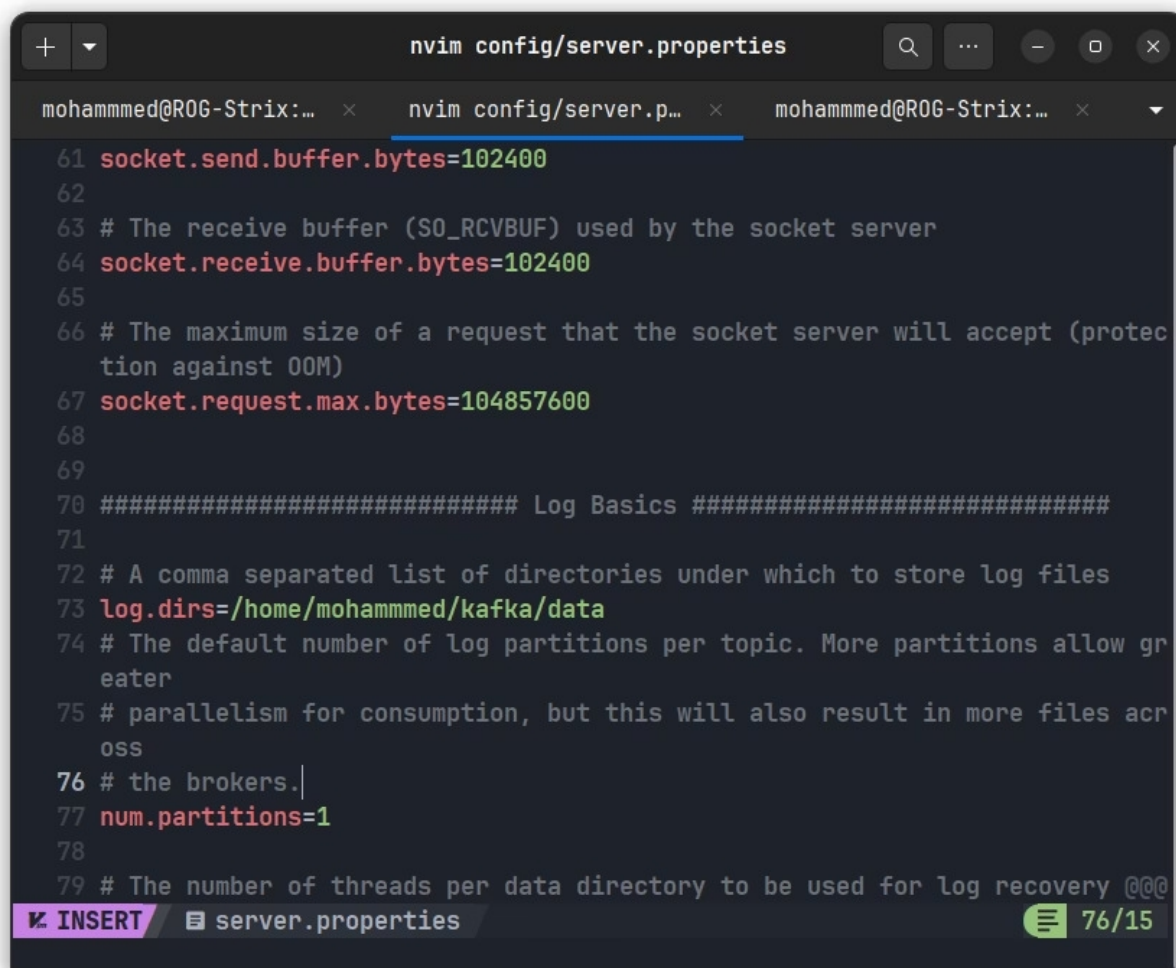
2. Configuration des variable d'environnement

A terminal window titled 'mohammed@ROG-Strix:~/Desktop/COUR...' with a tab for 'nvim env.zsh'. The terminal shows the configuration of environment variables for Kafka. Lines 37 and 38 set 'HBASE_HOME' and 'PATH' respectively. Lines 40-43 show the configuration for Kafka, including setting 'KAFKA_HOME' and updating 'PATH' to include the Kafka bin directory.

```
nvim env.zsh
mohammed@ROG-Strix:~/Desktop/COUR...
nvim env.zsh
37 export HBASE_HOME=$HOME/hbase
38 export PATH=$PATH:$HBASE_HOME/bin
39
40 # Kafka configuration
41 export KAFKA_HOME=$HOME/kafka
42 export PATH=$PATH:$KAFKA_HOME/bin |
43
```

3. Configuration du fichier de serveur config/server.properties

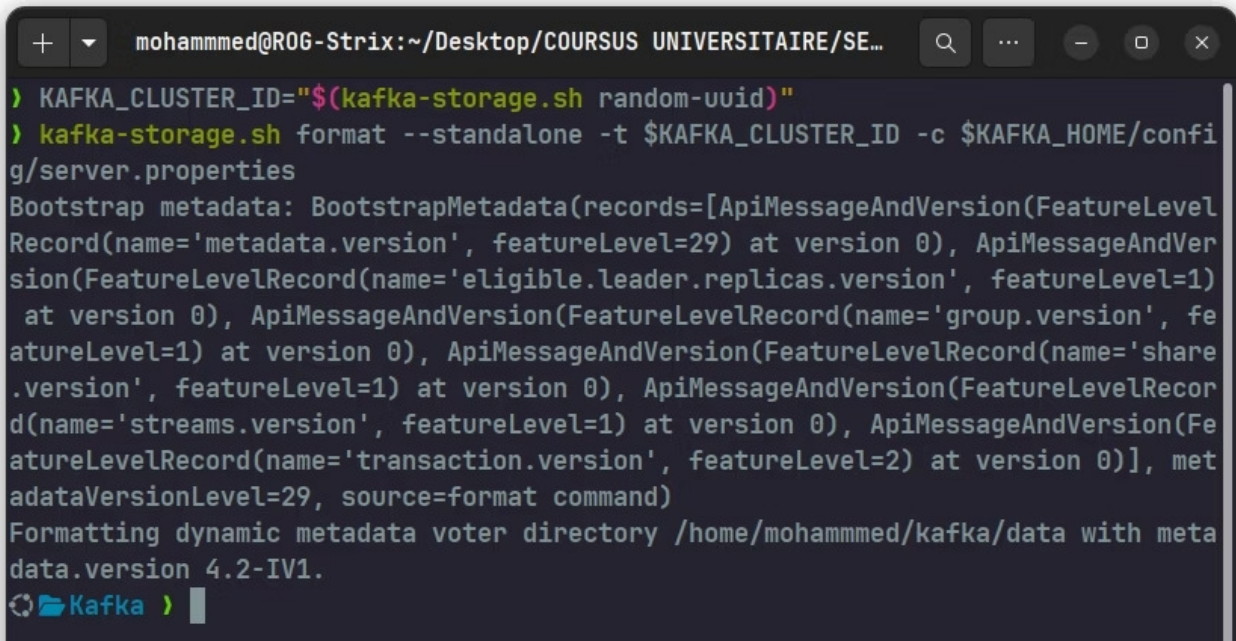
Le fichier `server.properties` constitue le principal fichier de configuration d'Apache Kafka. Dans cette étape, plusieurs paramètres sont ajustés afin de définir le comportement du serveur Kafka, notamment l'emplacement de stockage des données (`log.dirs`) ainsi que le nombre de partitions par défaut (`num.partitions`). Ces paramètres permettent d'organiser le stockage des messages et d'optimiser le traitement des données au sein du broker Kafka.



```
nvim config/server.properties
mohammed@ROG-Strix:... x nvim config/server.p... x mohammed@ROG-Strix:... x
61 socket.send.buffer.bytes=102400
62
63 # The receive buffer (SO_RCVBUF) used by the socket server
64 socket.receive.buffer.bytes=102400
65
66 # The maximum size of a request that the socket server will accept (protec
tion against OOM)
67 socket.request.max.bytes=104857600
68
69
70 ##### Log Basics #####
71
72 # A comma separated list of directories under which to store log files
73 log.dirs=/home/mohammed/kafka/data
74 # The default number of log partitions per topic. More partitions allow gr
eater
75 # parallelism for consumption, but this will also result in more files acr
oss
76 # the brokers.
77 num.partitions=1
78
79 # The number of threads per data directory to be used for log recovery @@@
❖ INSERT server.properties 76/15
```

4. Initialisation du cluster et formatage du stockage

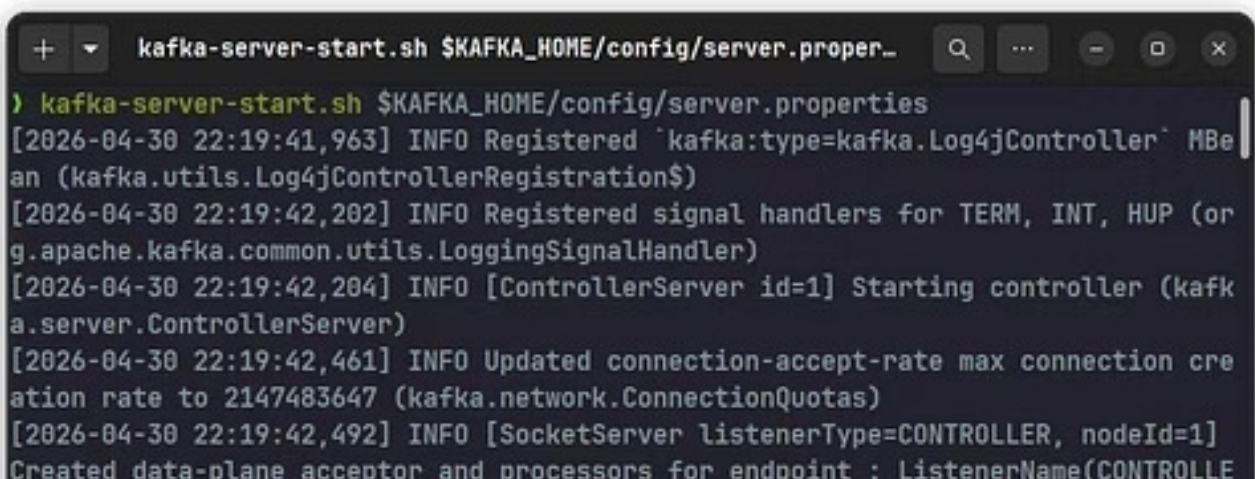
Depuis les versions récentes de Kafka, il est nécessaire d'initialiser le stockage du cluster avant le premier démarrage. Cette opération consiste à générer un identifiant unique du cluster (`cluster ID`) puis à formater le répertoire de stockage à l'aide de l'outil `kafka-storage.sh`. Cette étape permet à Kafka de préparer les métadonnées nécessaires au fonctionnement du broker.

A terminal window with a dark background. The title bar shows the user 'mohammed@ROG-Strix' and the path '~/Desktop/COURSUS UNIVERSITAIRE/SE...'. The terminal contains the following commands and output:

```
> KAFKA_CLUSTER_ID="$(kafka-storage.sh random-uuid)"
> kafka-storage.sh format --standalone -t $KAFKA_CLUSTER_ID -c $KAFKA_HOME/config/server.properties
Bootstrap metadata: BootstrapMetadata(records=[ApiMessageAndVersion(FutureLevelRecord(name='metadata.version', featureLevel=29) at version 0), ApiMessageAndVersion(FutureLevelRecord(name='eligible.leader.replicas.version', featureLevel=1) at version 0), ApiMessageAndVersion(FutureLevelRecord(name='group.version', featureLevel=1) at version 0), ApiMessageAndVersion(FutureLevelRecord(name='share.version', featureLevel=1) at version 0), ApiMessageAndVersion(FutureLevelRecord(name='streams.version', featureLevel=1) at version 0), ApiMessageAndVersion(FutureLevelRecord(name='transaction.version', featureLevel=2) at version 0)], metadataVersionLevel=29, source=format command)
Formatting dynamic metadata voter directory /home/mohammed/kafka/data with metadata.version 4.2-IV1.
Kafka >
```

5. Démarche du Broker Kafka

Une fois le stockage initialisé et le serveur configuré, le broker Kafka est démarré à l'aide du fichier de configuration `server.properties`. Le broker représente le composant principal de Kafka chargé de recevoir, stocker et distribuer les messages entre les producteurs (producers) et les consommateurs (consumers). Son démarrage permet de rendre le cluster opérationnel et prêt à traiter les flux de données.

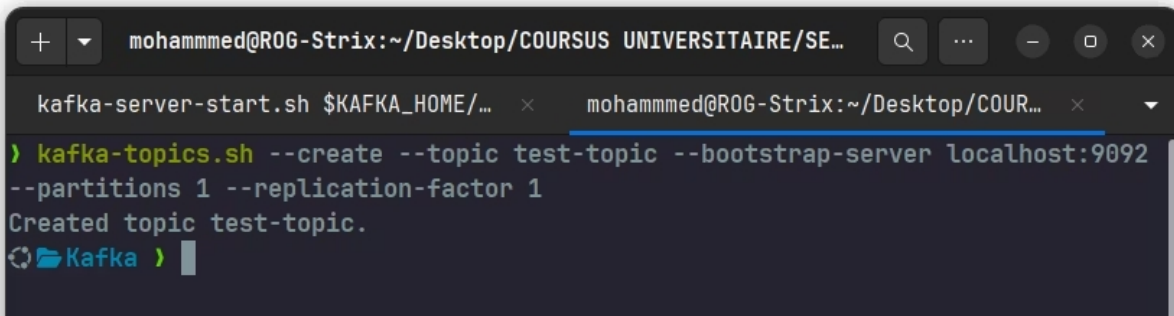
A terminal window with a dark background. The title bar shows the command 'kafka-server-start.sh \$KAFKA_HOME/config/server.properties'. The terminal contains the following commands and output:

```
> kafka-server-start.sh $KAFKA_HOME/config/server.properties
[2026-04-30 22:19:41,963] INFO Registered 'kafka:type=kafka.Log4jController' MBean (kafka.utils.Log4jControllerRegistration$)
[2026-04-30 22:19:42,202] INFO Registered signal handlers for TERM, INT, HUP (org.apache.kafka.common.utils.LoggingSignalHandler)
[2026-04-30 22:19:42,204] INFO [ControllerServer id=1] Starting controller (kafka.server.ControllerServer)
[2026-04-30 22:19:42,461] INFO Updated connection-accept-rate max connection creation rate to 2147483647 (kafka.network.ConnectionQuotas)
[2026-04-30 22:19:42,492] INFO [SocketServer listenerType=CONTROLLER, nodeId=1] Created data-plane acceptor and processors for endpoint : ListenerName(CONTROLLE
```


6. Création d'un topic

6.1 Création d'un Topic

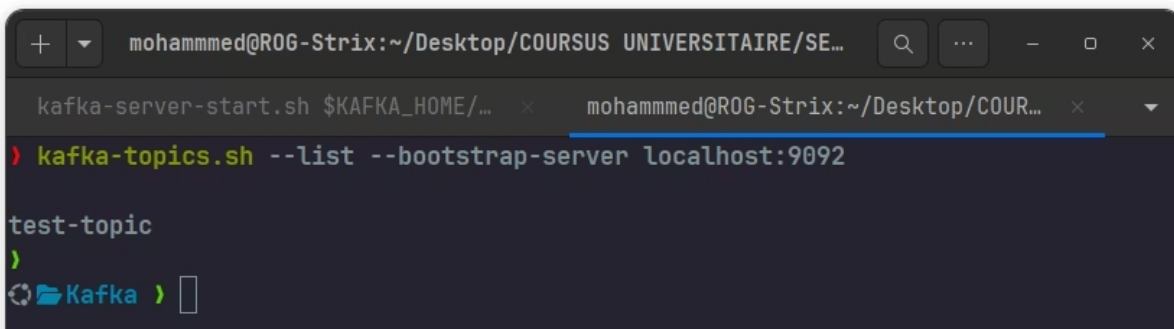
Dans Kafka, un **topic** représente un canal de communication permettant l'échange de messages entre les producteurs (producer) et les consommateurs (consumer). Dans cette étape, un topic nommé `test-topic` est créé à l'aide de la commande `kafka-topics.sh`. Ce topic est configuré avec une partition et un facteur de réplication égal à 1.

A terminal window with a dark background. The title bar shows the user 'mohammed@ROG-Strix' and the path '~/Desktop/COURSUS UNIVERSITAIRE/SE...'. There are two tabs: 'kafka-server-start.sh \$KAFKA_HOME/...' and 'mohammed@ROG-Strix:~/Desktop/COUR...'. The terminal shows the command `kafka-topics.sh --create --topic test-topic --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1` being executed. The output is `Created topic test-topic.` followed by a prompt `Kafka >`.

```
mohammed@ROG-Strix:~/Desktop/COURSUS UNIVERSITAIRE/SE...  
kafka-server-start.sh $KAFKA_HOME/... x mohammed@ROG-Strix:~/Desktop/COUR... x  
> kafka-topics.sh --create --topic test-topic --bootstrap-server localhost:9092  
--partitions 1 --replication-factor 1  
Created topic test-topic.  
Kafka >
```

6.2 Vérification des Topics

Après la création du topic, la commande `kafka-topics.sh --list` est utilisée afin d'afficher l'ensemble des topics présents dans le cluster Kafka. Cette vérification permet de confirmer la bonne création du topic.

A terminal window with a dark background. The title bar shows the user 'mohammed@ROG-Strix' and the path '~/Desktop/COURSUS UNIVERSITAIRE/SE...'. There are two tabs: 'kafka-server-start.sh \$KAFKA_HOME/...' and 'mohammed@ROG-Strix:~/Desktop/COUR...'. The terminal shows the command `kafka-topics.sh --list --bootstrap-server localhost:9092` being executed. The output is `test-topic` followed by a prompt `Kafka >`.

```
mohammed@ROG-Strix:~/Desktop/COURSUS UNIVERSITAIRE/SE...  
kafka-server-start.sh $KAFKA_HOME/... x mohammed@ROG-Strix:~/Desktop/COUR... x  
> kafka-topics.sh --list --bootstrap-server localhost:9092  
  
test-topic  
>  
Kafka >
```

6.3 Description du Topic

La commande `kafka-topics.sh --describe` permet d'obtenir des informations détaillées sur un topic, notamment son identifiant, son nombre de partitions, son facteur de réplication ainsi que les brokers responsables du stockage des données.

```
mohammed@ROG-Strix:~/Desktop/COURSUS UNIVERSITAIRE/SE...
kafka-server-start.sh $KAFKA_HOME/... x mohammed@ROG-Strix:~/Desktop/COUR... x
> kafka-topics.sh --describe --topic test-topic --bootstrap-server localhost:9092
Topic: test-topic      TopicId: RCivt0L4RpaRVPaUgv9JDg PartitionCount: 1      R
eplicationFactor: 1    Configs: min.insync.replicas=1,segment.bytes=1073741824
      Topic: test-topic      Partition: 0      Leader: 1      Replicas: 1      I
sr: 1      Elr:      LastKnownElr:
Kafka >
```

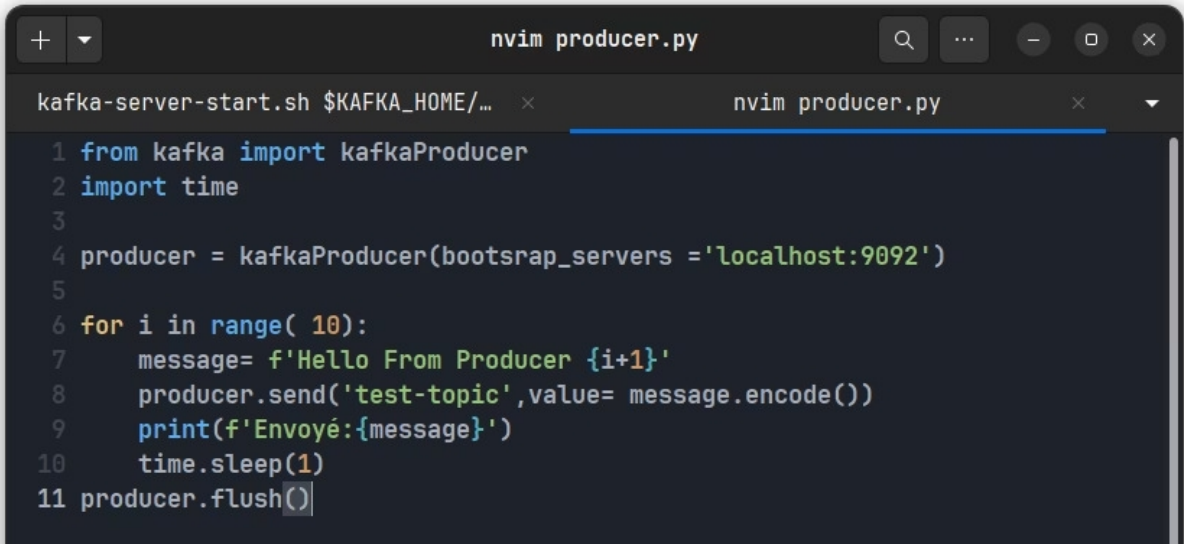
7. Installation des bibliothèques Kafka pour Python

Afin de permettre le développement d'applications Kafka en Python, les bibliothèques `kafka-python` et `confluent-kafka` sont installées à l'aide du gestionnaire de paquets `pip`. Ces bibliothèques fournissent les interfaces nécessaires pour créer des producteurs (`producer`) et des consommateurs (`consumer`) capables d'échanger des messages avec un cluster Kafka.

```
mohammed@ROG-Strix:~/Desktop/COURSUS UNIVERSITAIRE/SE...
kafka-server-start.sh $KAFKA_HOME/... x mohammed@ROG-Strix:~/Desktop/COUR... x
> pip install kafka-python confluent-kafka
Collecting kafka-python
  Downloading kafka_python-2.3.1-py2.py3-none-any.whl.metadata (9.5 kB)
Collecting confluent-kafka
  Downloading confluent_kafka-2.14.0-cp312-cp312-manylinux_2_28_x86_64.whl.metad
ata (32 kB)
Downloading kafka_python-2.3.1-py2.py3-none-any.whl (326 kB)
Downloading confluent_kafka-2.14.0-cp312-cp312-manylinux_2_28_x86_64.whl (4.0 MB)
----- 4.0/4.0 MB 813.6 kB/s 0:00:04
Installing collected packages: kafka-python, confluent-kafka
Successfully installed confluent-kafka-2.14.0 kafka-python-2.3.1
Kafka > took 8s kafka
```

8. Création d'un Producteur Kafka en Python

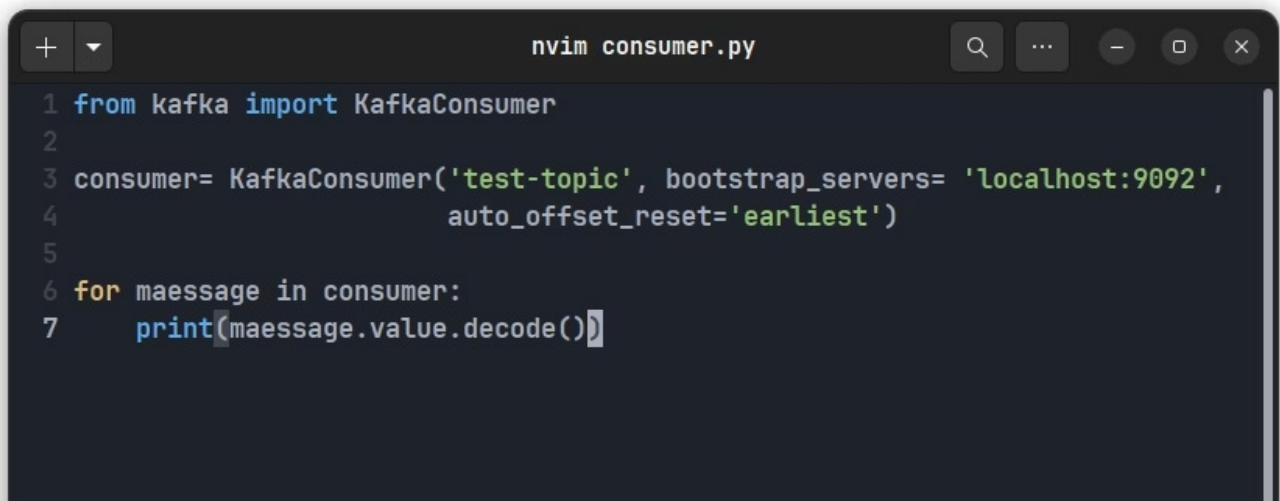
Un script Python nommé `producer.py` est développé afin de jouer le rôle de producteur Kafka. Ce programme établit une connexion avec le broker Kafka via l'adresse `localhost:9092`, puis envoie une série de messages vers le topic `test-topic`. Chaque message est transmis à intervalle régulier afin de simuler un flux de données en temps réel.

A screenshot of a terminal window with a dark background. The title bar shows 'nvim producer.py'. The code is written in Python and uses the kafka-python library. It imports kafkaProducer and time, then creates a producer with bootstrap_servers set to 'localhost:9092'. A loop runs 10 times, sending messages 'Hello From Producer {i+1}' to the 'test-topic' topic, with a 1-second delay between each message. The code ends with a flush call.

```
1 from kafka import kafkaProducer
2 import time
3
4 producer = kafkaProducer(bootstrap_servers = 'localhost:9092')
5
6 for i in range( 10):
7     message= f'Hello From Producer {i+1}'
8     producer.send('test-topic',value= message.encode())
9     print(f'Envoyé:{message}')
10    time.sleep(1)
11 producer.flush()
```

9. Création d'un Consommateur Kafka en Python

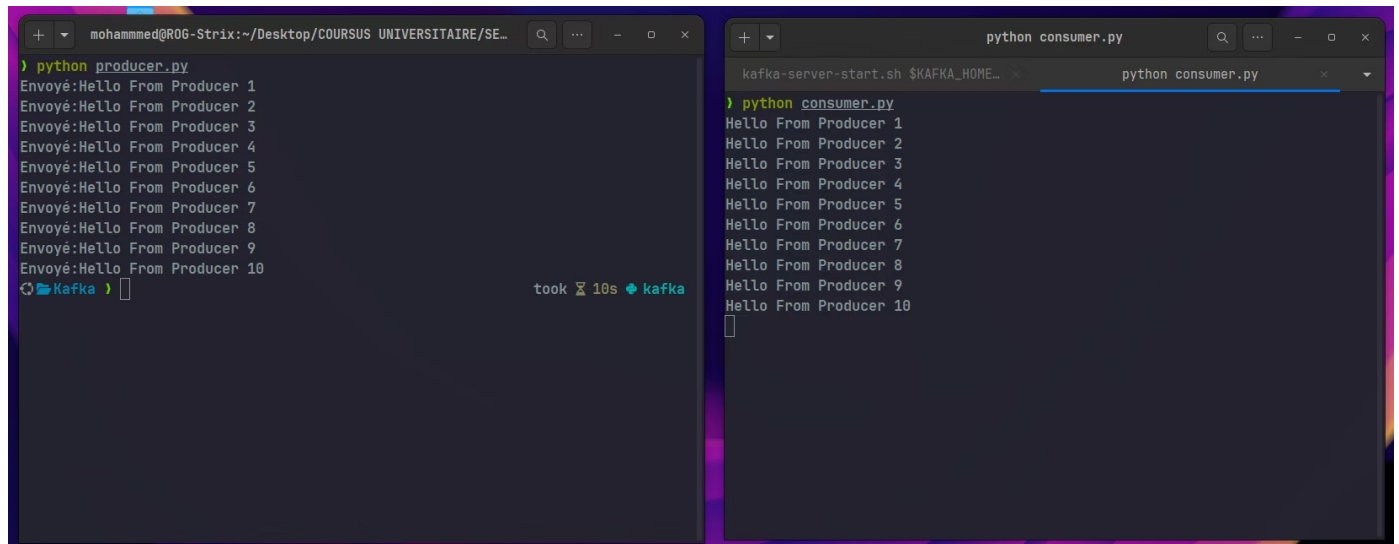
Après la mise en place du producteur Kafka, un script Python nommé `consumer.py` est développé afin de jouer le rôle de consommateur. Ce programme se connecte au broker Kafka via l'adresse `localhost:9092` et s'abonne au topic `test-topic`. Il écoute en continu les messages publiés par le producteur et les affiche dans le terminal dès leur réception.

A screenshot of a terminal window with a dark background. The title bar shows 'nvim consumer.py'. The code imports KafkaConsumer from the kafka module. It creates a consumer for the 'test-topic' topic with bootstrap_servers set to 'localhost:9092' and auto_offset_reset set to 'earliest'. A loop then iterates over the consumer, printing the decoded value of each message.

```
1 from kafka import KafkaConsumer
2
3 consumer= KafkaConsumer('test-topic', bootstrap_servers= 'localhost:9092',
4                          auto_offset_reset='earliest')
5
6 for maessage in consumer:
7     print(maessage.value.decode())
```


10. Résultat finale

Les scripts `producer.py` et `consumer.py` sont exécutés simultanément afin de valider le fonctionnement complet de l'architecture Kafka. Le producteur envoie une série de messages vers le topic `test-topic`, tandis que le consommateur récupère ces messages et les affiche en temps réel.



The image displays two terminal windows side-by-side, demonstrating the Kafka producer and consumer workflow.

Left Terminal (producer.py):

```
mohammed@ROG-Strix:~/Desktop/COURSUS UNIVERSITAIRE/SE...  
> python producer.py  
Envoyé:Hello From Producer 1  
Envoyé:Hello From Producer 2  
Envoyé:Hello From Producer 3  
Envoyé:Hello From Producer 4  
Envoyé:Hello From Producer 5  
Envoyé:Hello From Producer 6  
Envoyé:Hello From Producer 7  
Envoyé:Hello From Producer 8  
Envoyé:Hello From Producer 9  
Envoyé:Hello From Producer 10  
Kafka > took 10s kafka
```

Right Terminal (consumer.py):

```
python consumer.py  
kafka-server-start.sh $KAFKA_HOME...  
python consumer.py  
> python consumer.py  
Hello From Producer 1  
Hello From Producer 2  
Hello From Producer 3  
Hello From Producer 4  
Hello From Producer 5  
Hello From Producer 6  
Hello From Producer 7  
Hello From Producer 8  
Hello From Producer 9  
Hello From Producer 10
```